Name : **Kaviya. R**

Register number : **192524154**

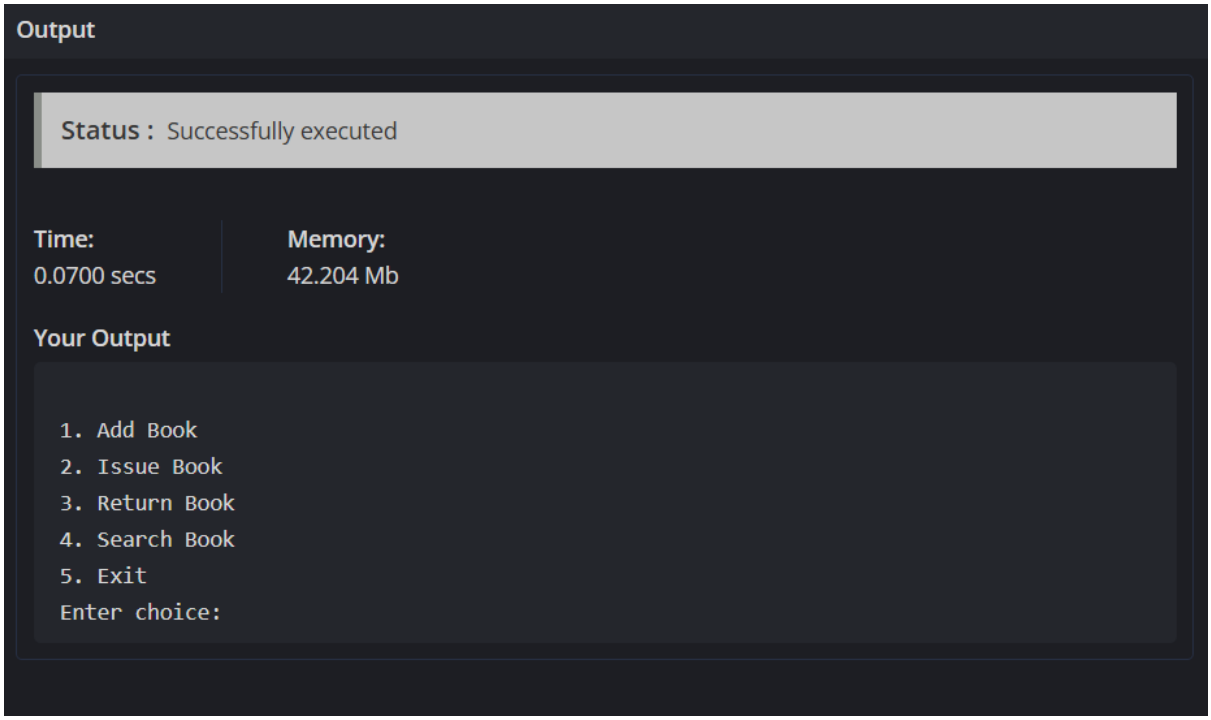
Course : **Java programming**

Course code : **CSA0904**

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of Objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

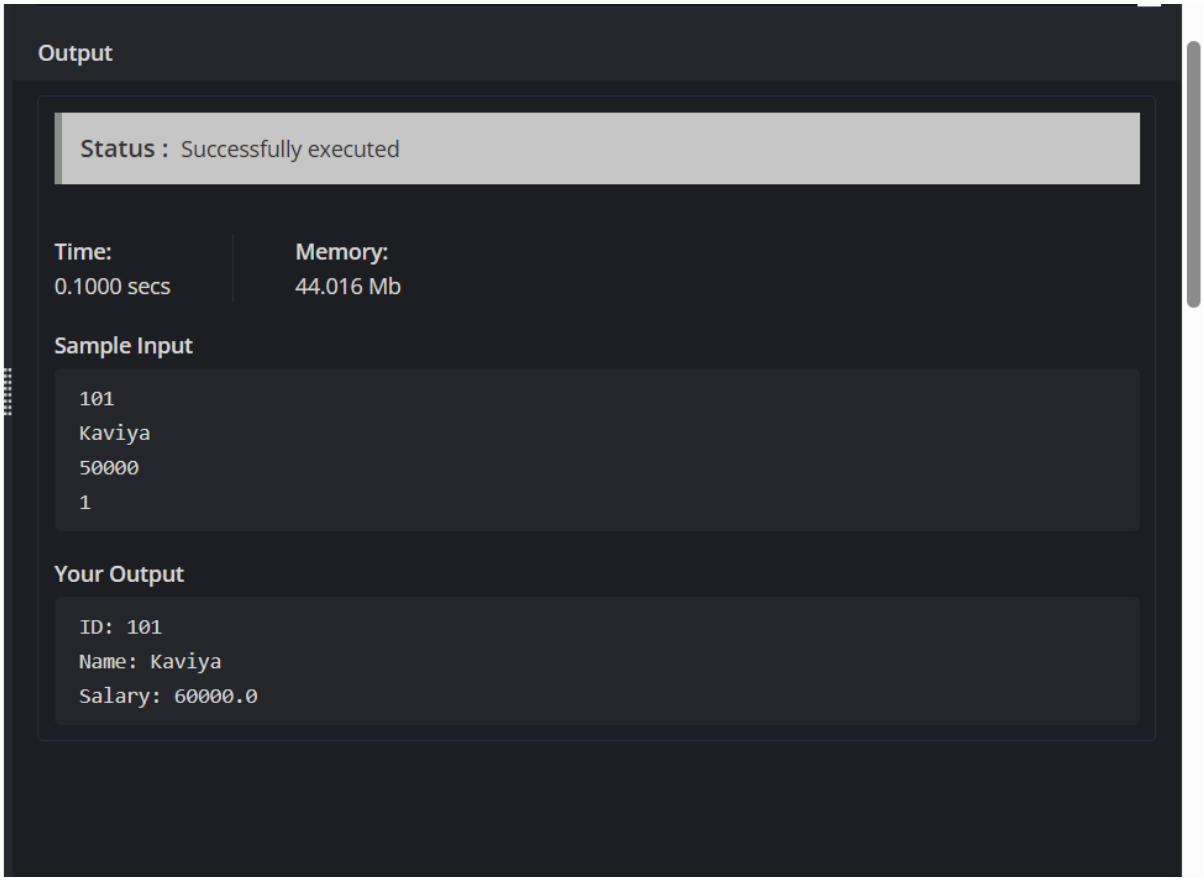
```java
import java.util.*;

class Book {
    private String title;
    private String author;
    private double price;
    private boolean issued;

    Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
        this.issued = false;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }

    public double getPrice() {
        return price;
    }

    public boolean isIssued() {
        return issued;
    }

    public void issue() {
        issued = true;
    }

    public void returnBook() {
        issued = false;
    }

    public void display() {
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Price: " + price);
        System.out.println("Status: " + (issued ? "Issued" : "Available"));
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Book> books = new ArrayList<>();

        while (true) {
            System.out.println("\n1. Add Book");
            System.out.println("2. Issue Book");
            System.out.println("3. Return Book");
            System.out.println("4. Search Book");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");

            if (!sc.hasNextInt())
                break;

            int choice = sc.nextInt();
            sc.nextLine();

            if (choice == 1) {
                System.out.print("Enter title: ");
                String title = sc.nextLine();

                System.out.print("Enter author: ");
                String author = sc.nextLine();

                System.out.print("Enter price: ");
                double price = sc.nextDouble();
                sc.nextLine();

                books.add(new Book(title, author, price));
                System.out.println("Book added successfully.");
            }

            else if (choice == 2) {
                System.out.print("Enter book title: ");
                String title = sc.nextLine();

                boolean found = false;

                for (Book book : books) {
                    if (book.getTitle().equalsIgnoreCase(title)) {
                        found = true;

                        if (!book.isIssued()) {
                            book.issue();
                            System.out.println("Book issued successfully.");
                        } else {
                            System.out.println("Book is already issued.");
                        }

                        break;
                    }
                }

                if (!found)
                    System.out.println("Book not found.");
            }

            else if (choice == 3) {
                System.out.print("Enter book title: ");
                String title = sc.nextLine();

                boolean found = false;

                for (Book book : books) {
                    if (book.getTitle().equalsIgnoreCase(title)) {
                        found = true;

                        if (book.isIssued()) {
                            book.returnBook();
                            System.out.println("Book returned successfully.");
                        } else {
                            System.out.println("Book was not issued.");
                        }

                        break;
                    }
                }

                if (!found)
                    System.out.println("Book not found.");
            }

            else if (choice == 4) {
                System.out.print("Enter book title: ");
                String title = sc.nextLine();

                boolean found = false;

                for (Book book : books) {
                    if (book.getTitle().equalsIgnoreCase(title)) {
                        book.display();
                        found = true;
                        break;
                    }
                }

                if (!found)
                    System.out.println("Book not found.");
            }

            else if (choice == 5) {
                System.out.println("Exiting...");
                break;
            }

            else {
                System.out.println("Invalid choice.");
            }
        }

        sc.close();
    }
}
```



2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

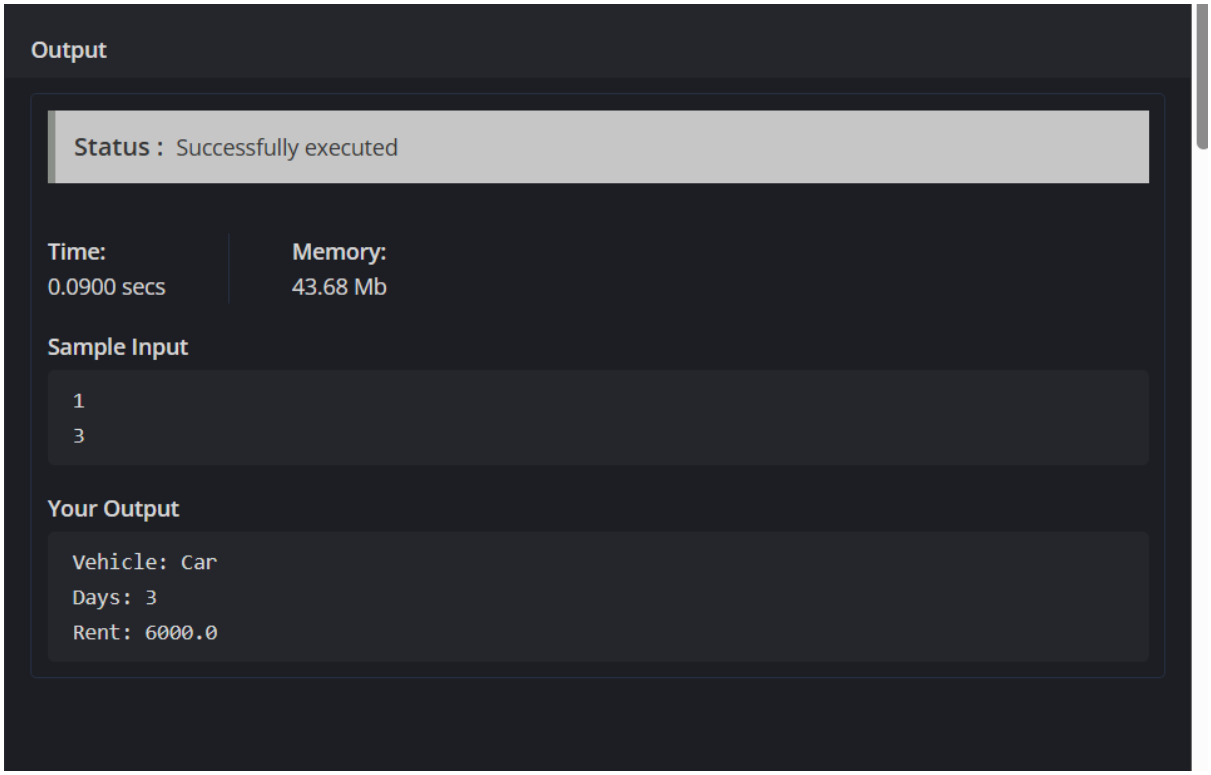
```java
import java.util.*;

class Employee {
    int id;
    String name;
    double basicSalary;

    Employee(int id, String name, double basicSalary) {
        this.id = id;
        this.name = name;
        this.basicSalary = basicSalary;
    }

    double calculateSalary() {
        return basicSalary;
    }

    void display() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Salary: " + calculateSalary());
    }
}

class PermanentEmployee extends Employee {
    PermanentEmployee(int id, String name, double basicSalary) {
        super(id, name, basicSalary);
    }

    double calculateSalary() {
        return basicSalary + basicSalary * 0.20;
    }
}

class ContractEmployee extends Employee {
    ContractEmployee(int id, String name, double basicSalary) {
        super(id, name, basicSalary);
    }

    double calculateSalary() {
        return basicSalary;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        if (!sc.hasNextInt())
            return;

        int id = sc.nextInt();

        if (!sc.hasNext())
            return;

        String name = sc.next();

        if (!sc.hasNextDouble())
            return;

        double salary = sc.nextDouble();

        if (!sc.hasNextInt())
            return;

        int type = sc.nextInt();

        Employee employee;

        if (type == 1)
            employee = new PermanentEmployee(id, name, salary);
        else
            employee = new ContractEmployee(id, name, salary);

        employee.display();
    }
}
```



3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

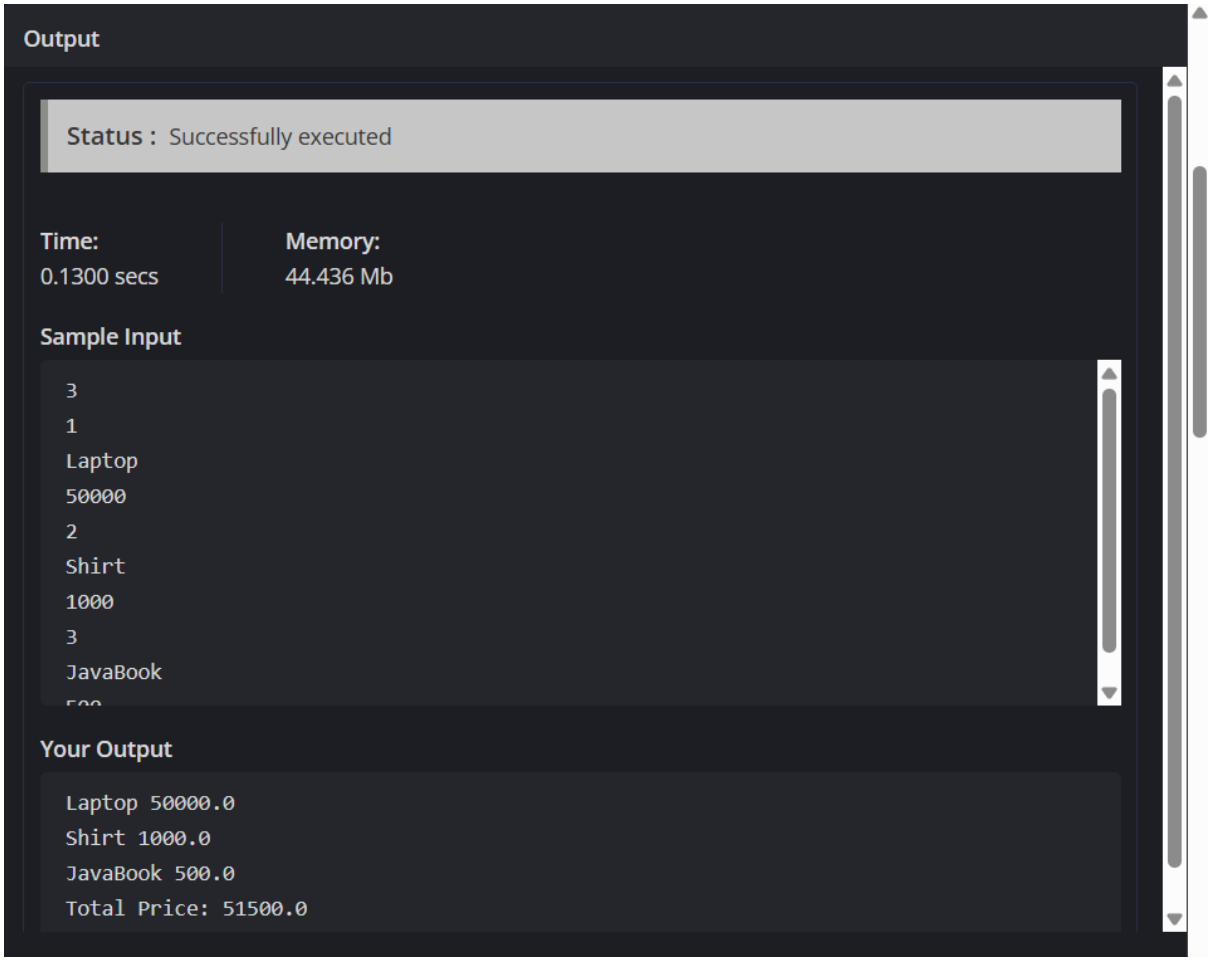
```java
import java.util.*;

class Vehicle {
    String name;
    double rentPerDay;

    Vehicle(String name, double rentPerDay) {
        this.name = name;
        this.rentPerDay = rentPerDay;
    }

    double calculateRent(int days) {
        return rentPerDay * days;
    }

    void display(int days) {
        System.out.println("Vehicle: " + name);
        System.out.println("Days: " + days);
        System.out.println("Rent: " +
calculateRent(days));
}

class Car extends Vehicle {
    Car(String name, double rentPerDay) {
        super(name, rentPerDay);
    }

    double calculateRent(int days) {
        return rentPerDay * days;
    }
}

class Bike extends Vehicle {
    Bike(String name, double rentPerDay) {
        super(name, rentPerDay);
    }

    double calculateRent(int days) {
        return rentPerDay * days;
    }
}

class Bus extends Vehicle {
    Bus(String name, double rentPerDay) {
        super(name, rentPerDay);
    }

    double calculateRent(int days) {
        return rentPerDay * days;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        if (!sc.hasNextInt())
            return;

        int type = sc.nextInt();

        if (!sc.hasNextInt())
            return;

        int days = sc.nextInt();

        Vehicle vehicle;

        if (type == 1)
            vehicle = new Car("Car", 2000);
        else if (type == 2)
            vehicle = new Bike("Bike", 800);
        else
            vehicle = new Bus("Bus", 5000);

        vehicle.display(days);
    }
}
```



4. Online Shopping Cart using Interfaces

Create an interface Product with methods getPrice() and displayProduct(). Implement this interface in classes Electronics, Clothing, and Books. Create a shopping cart that stores different product types and calculates the total price using interface references. Also, construct suitable test cases to validate product addition and total price calculation. BTL: BL4 (Analyze)

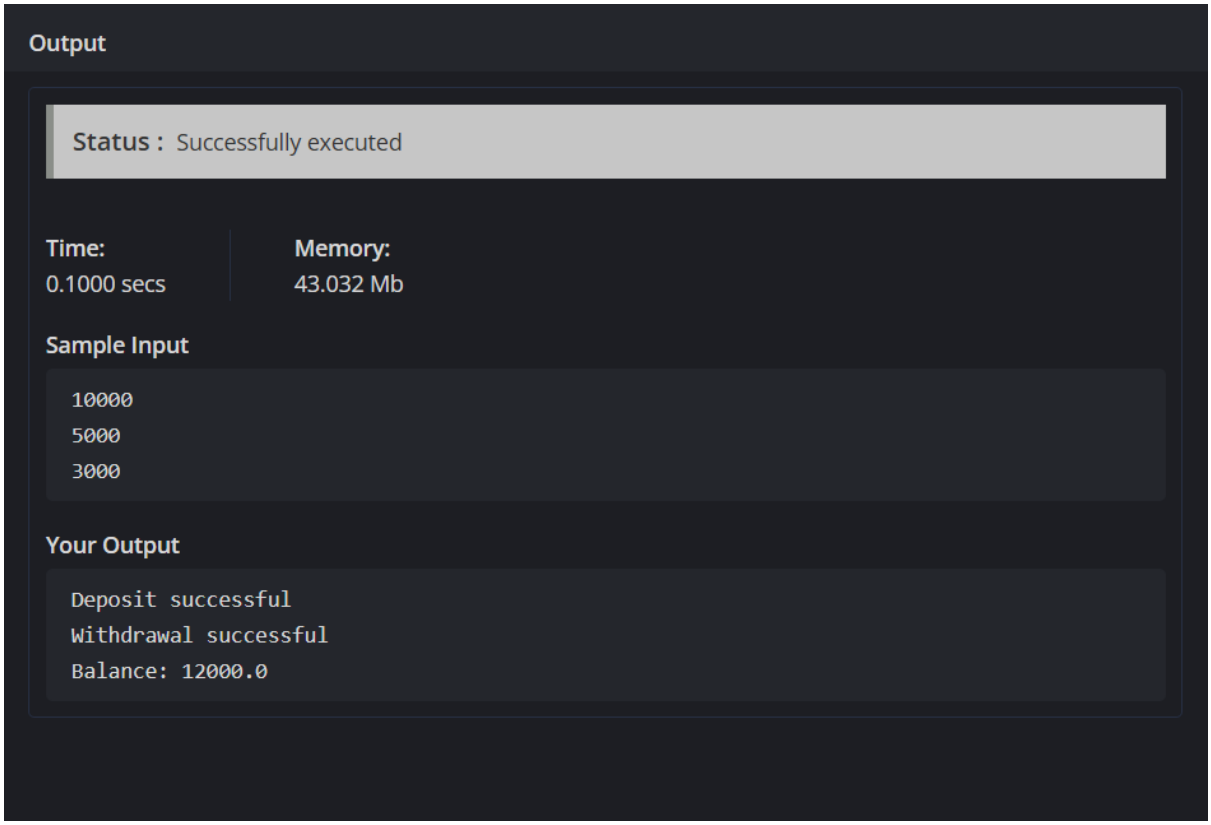
```java
import java.util.*;

interface Product {
    double getPrice();
    void displayProduct();
}

class Electronics implements Product {
    String name;
    double price;

    Electronics(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    public void displayProduct() {
        System.out.println(name + " " + price);
    }
}

class Clothing implements Product {
    String name;
    double price;

    Clothing(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    public void displayProduct() {
        System.out.println(name + " " + price);
    }
}

class Books implements Product {
    String name;
    double price;

    Books(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    public void displayProduct() {
        System.out.println(name + " " + price);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        ArrayList<Product> cart = new ArrayList<>();

        if (!sc.hasNextInt())
            return;

        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            if (!sc.hasNextInt())
                return;

            int type = sc.nextInt();

            if (!sc.hasNext())
                return;

            String name = sc.next();

            if (!sc.hasNextDouble())
                return;

            double price = sc.nextDouble();

            if (type == 1)
                cart.add(new Electronics(name, price));
            else if (type == 2)
                cart.add(new Clothing(name, price));
            else if (type == 3)
                cart.add(new Books(name, price));
        }

        double total = 0;

        for (Product product : cart) {
            product.displayProduct();
            total += product.getPrice();
        }

        System.out.println("Total Price: " + total);
    }
}
```



5. Bank Account Management using Exception Handling

Create a BankAccount class with methods for deposit and withdrawal. Use exception handling to manage insufficient balance and invalid transaction amounts. Create custom exceptions such as InsufficientBalanceException and InvalidAmountException. Also, construct suitable test cases to validate deposit and withdrawal operations. BTL: BL3 (Apply)

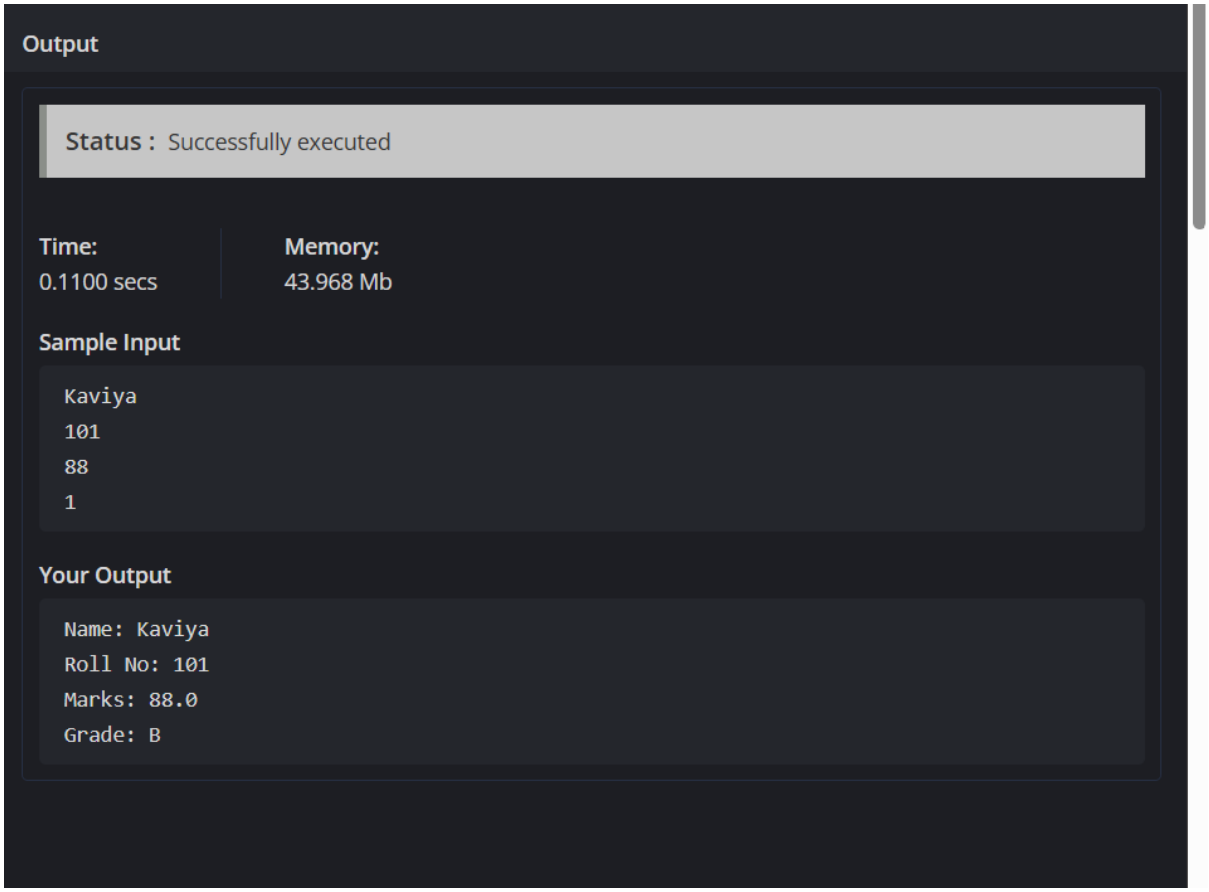
```java
import java.util.*;

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

class InvalidAmountException extends Exception {
    InvalidAmountException(String message) {
        super(message);
    }
}

class BankAccount {
    double balance;

    BankAccount(double balance) {
        this.balance = balance;
    }

    void deposit(double amount) throws InvalidAmountException {
        if (amount <= 0)
            throw new InvalidAmountException("Invalid deposit amount");

        balance += amount;
        System.out.println("Deposit successful");
    }

    void withdraw(double amount)
            throws InvalidAmountException, InsufficientBalanceException {

        if (amount <= 0)
            throw new InvalidAmountException("Invalid withdrawal amount");

        if (amount > balance)
            throw new InsufficientBalanceException("Insufficient balance");

        balance -= amount;
        System.out.println("Withdrawal successful");
    }

    void displayBalance() {
        System.out.println("Balance: " + balance);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        if (!sc.hasNextDouble())
            return;

        double initialBalance = sc.nextDouble();

        BankAccount account = new BankAccount(initialBalance);

        if (!sc.hasNextDouble())
            return;

        double depositAmount = sc.nextDouble();

        try {
            account.deposit(depositAmount);
        } catch (InvalidAmountException e) {
            System.out.println(e.getMessage());
        }

        if (!sc.hasNextDouble())
            return;

        double withdrawAmount = sc.nextDouble();

        try {
            account.withdraw(withdrawAmount);
        } catch (InvalidAmountException | InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }

        account.displayBalance();
    }
}
```



6. Student Grade Management using Abstract Classes

Create an abstract class Student with an abstract method calculateGrade(). Implement this method in subclasses UndergraduateStudent and PostgraduateStudent based on their marks. Display the student details and grade using abstraction. Also, construct suitable test cases to validate grade calculation for different student categories. BTL: BL3 (Apply)

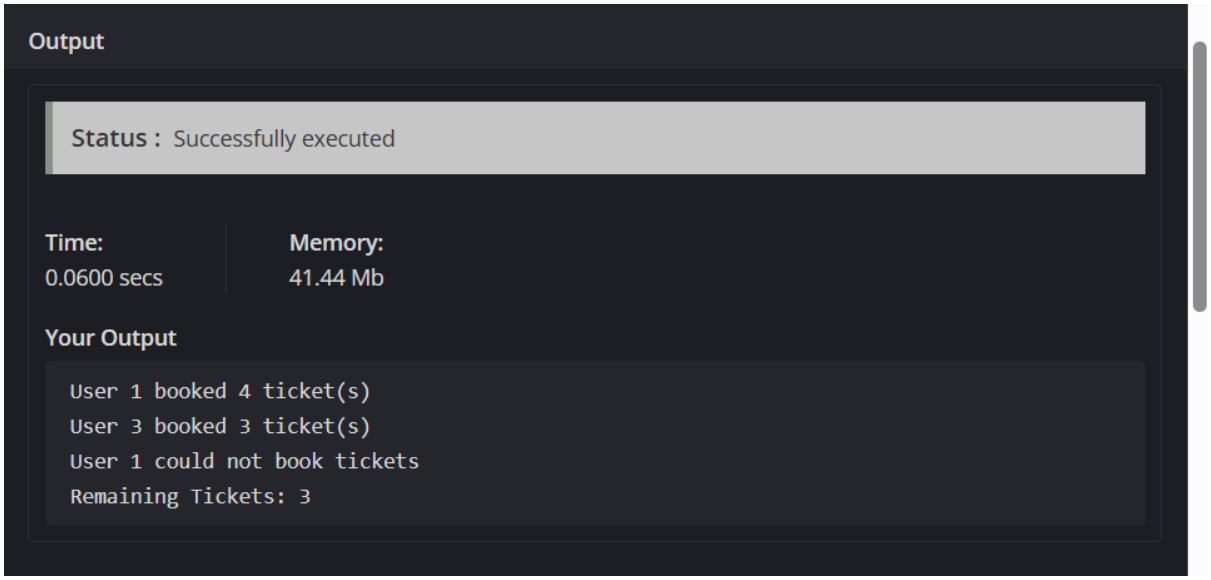
```java
import java.util.*;

abstract class Student {
    String name;
    int rollNo;
    double marks;

    Student(String name, int rollNo, double marks) {
        this.name = name;
        this.rollNo = rollNo;
        this.marks = marks;
    }

    abstract char calculateGrade();

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Marks: " + marks);
        System.out.println("Grade: " + calculateGrade());
    }
}

class UndergraduateStudent extends Student {
    UndergraduateStudent(String name, int rollNo, double marks) {
        super(name, rollNo, marks);
    }

    char calculateGrade() {
        if (marks >= 90)
            return 'A';
        else if (marks >= 75)
            return 'B';
        else if (marks >= 60)
            return 'C';
        else if (marks >= 50)
            return 'D';
        else
            return 'F';
    }
}

class PostgraduateStudent extends Student {
    PostgraduateStudent(String name, int rollNo, double marks) {
        super(name, rollNo, marks);
    }

    char calculateGrade() {
        if (marks >= 85)
            return 'A';
        else if (marks >= 70)
            return 'B';
        else if (marks >= 55)
            return 'C';
        else if (marks >= 50)
            return 'D';
        else
            return 'F';
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        if (!sc.hasNext())
            return;

        String name = sc.next();

        if (!sc.hasNextInt())
            return;

        int rollNo = sc.nextInt();

        if (!sc.hasNextDouble())
            return;

        double marks = sc.nextDouble();

        if (!sc.hasNextInt())
            return;

        int type = sc.nextInt();

        Student student;

        if (type == 1)
            student = new UndergraduateStudent(name, rollNo, marks);
        else
            student = new PostgraduateStudent(name, rollNo, marks);

        student.display();
    }
}
```



7. Multithreaded Ticket Booking System

Create a ticket booking system using multithreading where multiple users try to book tickets simultaneously. Use synchronization to prevent overbooking and ensure thread safety. Display booking details and the remaining ticket count. Also, construct suitable test cases to validate concurrent booking scenarios. BTL: BL4 (Analyze)

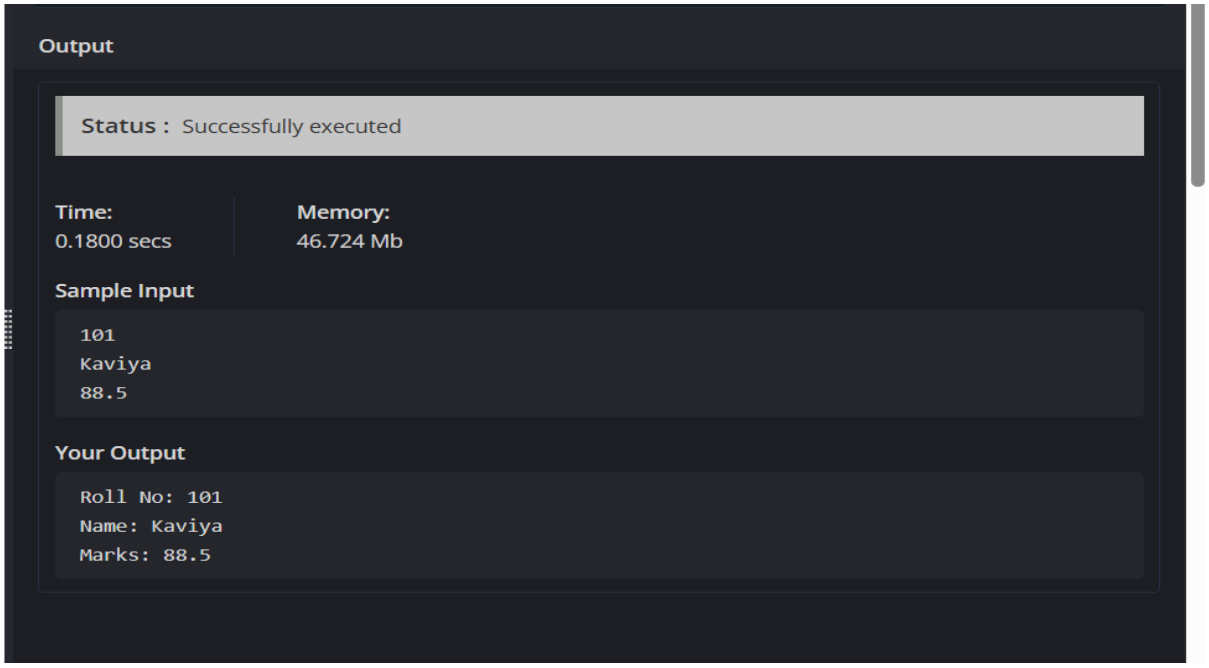
```java
class TicketBooking {
    private int tickets;

    TicketBooking(int tickets) {
        this.tickets = tickets;
    }

    synchronized void bookTicket(String user, int count) {
        if (count <= tickets) {
            tickets -= count;
            System.out.println(user + " booked " + count + "
ticket(s)");
        } else {
            System.out.println(user + " could not book tickets");
        }
    }

    void displayRemainingTickets() {
        System.out.println("Remaining Tickets: " + tickets);
    }
}

class User extends Thread {
    TicketBooking booking;
    String user;
    int tickets;

    User(TicketBooking booking, String user, int tickets) {
        this.booking = booking;
        this.user = user;
        this.tickets = tickets;
    }

    public void run() {
        booking.bookTicket(user, tickets);
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        TicketBooking booking = new TicketBooking(10);

        User user1 = new User(booking, "User 1", 4);
        User user2 = new User(booking, "User 1", 5);
        User user3 = new User(booking, "User 3", 3);

        user1.start();
        user2.start();
        user3.start();

        user1.join();
        user2.join();
        user3.join();

        booking.displayRemainingTickets();
    }
}
```



8. File Handling and Serialization

Create a Java program to serialize and deserialize student objects using file handling. Store student details in a file and retrieve them later. Use ObjectOutputStream and ObjectInputStream to perform serialization and deserialization. Also, construct suitable test cases to validate the storage and retrieval of student objects. BTL: BL3 (Apply)

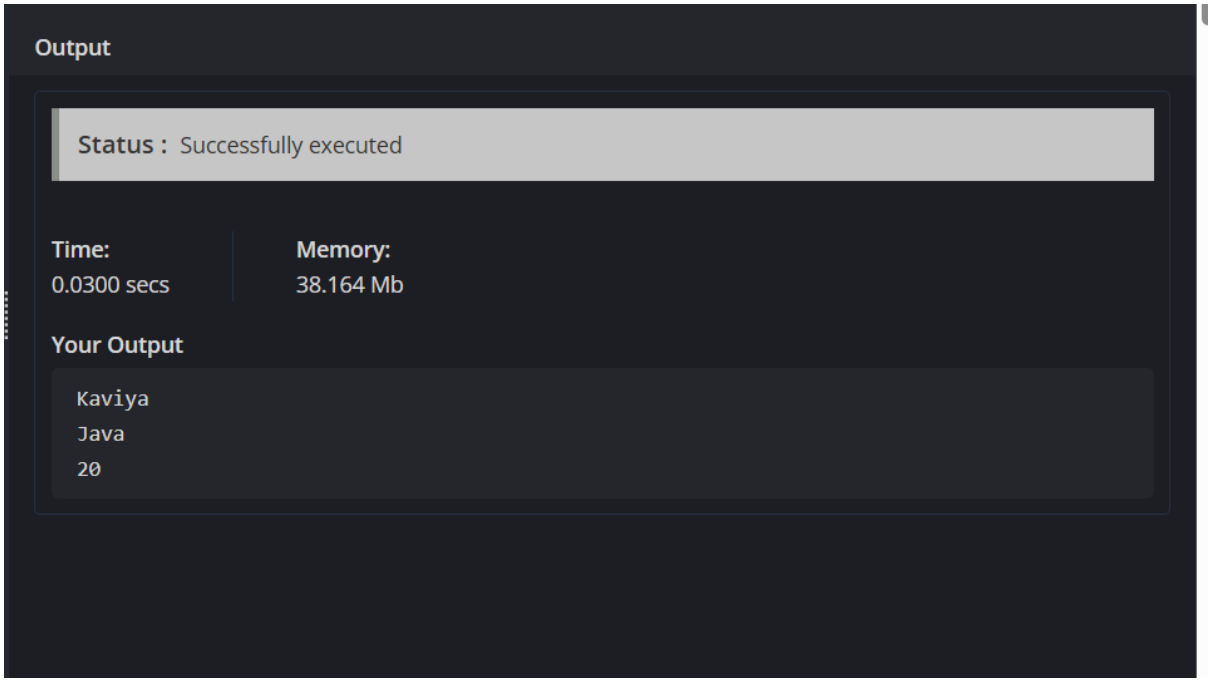
```java
import java.io.*;
import java.util.*;

class Student implements Serializable {
    int rollNo;
    String name;
    double marks;

    Student(int rollNo, String name, double marks) {
        this.rollNo = rollNo;
        this.name = name;
        this.marks = marks;
    }

    void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
        System.out.println("Marks: " + marks);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        if (!sc.hasNextInt())
            return;

        int rollNo = sc.nextInt();

        if (!sc.hasNext())
            return;

        String name = sc.next();

        if (!sc.hasNextDouble())
            return;

        double marks = sc.nextDouble();

        Student student = new Student(rollNo, name, marks);

        try {
            ObjectOutputStream out =
                    new ObjectOutputStream(new
FileOutputStream("student.txt"));
            out.writeObject(student);
            out.close();

            ObjectInputStream in =
                    new ObjectInputStream(new FileInputStream("student.txt"));

            Student s = (Student) in.readObject();
            in.close();

            s.display();

        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```



9. Generic Collection Framework

Create a generic class Storage<T> that can store and manage different types of objects. Implement methods to add, remove, and display elements. Use the collection framework to store objects of different data types. Also, construct suitable test cases to validate the generic collection operations. BTL: BL3 (Apply)

```java
import java.util.*;

class Storage<T> {
    ArrayList<T> list = new ArrayList<>();

    void add(T item) {
        list.add(item);
    }

    void remove(T item) {
        list.remove(item);
    }

    void display() {
        for (T item : list) {
            System.out.println(item);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Storage<String> names = new Storage<>();
        names.add("Jesika");
        names.add("Java");
        names.display();

        Storage<Integer> numbers = new Storage<>
();      numbers.add(10);
        numbers.add(20);
        numbers.remove(10);
        numbers.display();
    }
}
```



10. Mini Project: Integrated Java Application

Develop a mini project that integrates OOP concepts, inheritance, polymorphism, interfaces, exception handling, multithreading, file handling, serialization, and generic collections. The application should demonstrate the practical use of these concepts in a real-world scenario. Also, construct suitable test cases to validate the complete application. BTL: BL5 (Create)

```java
import java.io.*;
import java.util.*;

interface Payable {
    double calculateSalary();
}

abstract class Employee implements Serializable {
    int id;
    String name;
    double salary;

    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    abstract void display();
}

class PermanentEmployee extends Employee implements Payable {
    PermanentEmployee(int id, String name, double salary) {
        super(id, name, salary);
    }

    public double calculateSalary() {
        return salary + salary * 0.20;
    }

    void display() {
        System.out.println("Permanent Employee");
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Salary: " + calculateSalary());
    }
}

class ContractEmployee extends Employee implements Payable {
    ContractEmployee(int id, String name, double salary) {
        super(id, name, salary);
    }

    public double calculateSalary() {
        return salary;
    }

    void display() {
        System.out.println("Contract Employee");
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Salary: " + calculateSalary());
    }
}

class EmployeeStorage<T> {
    ArrayList<T> list = new ArrayList<>();

    void add(T item) {
        list.add(item);
    }

    void display() {
        for (T item : list) {
            System.out.println(item);
        }
    }
}

class SalaryThread extends Thread {
    Employee employee;

    SalaryThread(Employee employee) {
        this.employee = employee;
    }

    public void run() {
        employee.display();
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        if (!sc.hasNextInt())
            return;

        int id = sc.nextInt();

        if (!sc.hasNext())
            return;

        String name = sc.next();

        if (!sc.hasNextDouble())
            return;

        double salary = sc.nextDouble();

        if (!sc.hasNextInt())
            return;

        int type = sc.nextInt();

        Employee employee;

        if (type == 1)
            employee = new PermanentEmployee(id, name, salary);
        else
            employee = new ContractEmployee(id, name, salary);

        EmployeeStorage<Employee> storage = new EmployeeStorage<>();
        storage.add(employee);

        try {
            ObjectOutputStream out =
                    new ObjectOutputStream(new
FileOutputStream("employee.dat"));
            out.writeObject(employee);
            out.close();

            ObjectInputStream in =
                    new ObjectInputStream(new FileInputStream("employee.dat"));

            Employee savedEmployee = (Employee) in.readObject();
            in.close();

            SalaryThread thread = new SalaryThread(savedEmployee);
            thread.start();
            thread.join();

        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output

Status : Successfully executed

Time:	Memory:
0.1700 secs	46.692 Mb

Sample Input

```
101
Kaviya
50000
1
```

Your Output

```
Permanent Employee
ID: 101
Name: Kaviya
Salary: 60000.0
```